

G-computation formula

David M. Vock

Section 1

G-computation Algorithm

Model Entire Longitudinal Process

- In theory, under appropriate assumptions (consistency and sequential randomization assignment), then

$$P(Y^{\bar{a}_M} = y, L_M^{\bar{a}_{M-1}} = l_M, \dots, L_0 = l_0) = \quad (1)$$

$$P(Y = y | \bar{L}_M = \bar{l}_M, \bar{A}_M = \bar{a}_M) \quad (2)$$

$$\times P(L_M = l_M | \bar{L}_{M-1} = \bar{l}_{M-1}, \bar{A}_{M-1} = \bar{a}_{M-1}) \quad (3)$$

$$\times \vdots \quad (4)$$

$$\times P(L_0 = l_0) \quad (5)$$

- That is, the distribution of the potential outcome (and any summary measure of the this distribution like the mean) is identified from the observed data under appropriate assumptions

(Parametric) G-computation Algorithm

- NOTE: note to be confused with G-estimation (yes the people smart enough to come up with these techniques gave them very similar names)
- To do the (Parametric) G-computation algorithm in practice we could proceed as follows.

① Posit models for the conditional densities

$$P_{L_j|\bar{L}_{j-1},\bar{A}_{j-1}}(l_j|\bar{l}_{j-1},\bar{a}_{j-1};\psi_j) \text{ for } j = 1, \dots, M$$

$$P_{L_0}(l_0;\psi_0)$$

$$P_{Y|\bar{L}_M,\bar{A}_M}(y|\bar{l}_M,\bar{a}_M;\psi_{M+1})$$

in terms of the parameters $(\psi_0, \dots, \psi_{M+1})$ where ψ_j may be a vector of parameters. Note that some of these parameters could be shared across different time points

(Parametric) G-computation Algorithm

- ② Obtain parameter estimates using least squares/maximum likelihood using standard software
- ③ Integrating out the L 's analytically would likely be prohibitive. Therefore, we could approximate the distribution of $Y^{\bar{a}}$ for any $\bar{a}$ of interest using Monte Carlo integration. Specifically, for $r = 1, \dots, K$ (number of simulations), fix $\bar{a}$ and then
 - i) generate a random l_{0r} from $P_{L_0}(l_0; \hat{\psi}_0)$
 - ii) generate a random l_{1r} from $P_{L_1|L_0, A_0}(l_1|l_{0r}, a_0; \hat{\psi}_1)$
 - iii) generate a random l_{jr} from $P_{L_j|\bar{L}_{j-1}, \bar{A}_{j-1}}(l_j|\bar{l}_{j-1,r}, \bar{a}_{j-1}; \hat{\psi}_j)$ for $j = 2, \dots, M$
 - iv) generate a random y_r from $P_{Y|\bar{L}_M, \bar{A}_M}(y|\bar{l}_{M,r}, \bar{a}_M; \hat{\psi}_{M+1})$

(Parametric) G-computation Algorithm

- The y_r for $r = 1, \dots, K$ derived this way represent random draws from the marginal distribution of $Y^{\bar{a}}$.
- For example, we could estimate $E\{Y^{\bar{a}}\}$ by $\frac{1}{K} \sum_{r=1}^K y_r$
- We are then able to compare different treatment combinations in this fashion
- Standard errors could be estimated using the bootstrap. Note that there are two “Monte Carlo” steps here - for the bootstrap and the integration

G computation Overview

- Essentially, we are trying to estimate the distribution of some outcome by modeling the entire longitudinal covariate process
- This requires fitting a whole bunch of models each of which has to be “right” in order to consistently estimate the distribution of the response under different treatment assignment patterns
- Note that we actually have to correctly specify the correct conditional distribution (not just the conditional mean) for each AND remember L_j could be multidimensional
- For this reason, this approach has not been preferred in the literature (and frequently go by other names like “micro simulation”)
- Note that we do NOT have to model the treatment assignment

Modeling Assumptions

- $lCD4_0 \sim N(\mu_0, \sigma_0^2)$
- $lCD4_j | \bar{R}_j, \bar{lCD4}_{j-1}, \bar{A}_{j-1} \sim N(\beta_0 + \beta_1 lCD4_{j-1} + \beta_2 A_{j-1} + \beta_3 R_j + \beta_4 lCD4_{j-1} A_{j-1} + \beta_5 lCD4_{j-1} R_j + \beta_6 A_{j-1} R_j + \beta_7 lCD4_{j-1} A_{j-1} R_j, \sigma^2)$
- $P(R_j = 1 | \bar{R}_{j-1}, \bar{lCD4}_{j-1}, \bar{A}_{j-1}) = \frac{\gamma_0 + \gamma_1 cum(\bar{A}_{j-1})}{1 + \gamma_0 + \gamma_1 cum(\bar{A}_{j-1})}$ if $R_{j-1} = 0$ and $cum(\bar{A}_{j-1}) > 0$, $= 1$ if $R_{j-1} = 1$ and $= 0$ if $cum(\bar{A}_{j-1}) = 0$
- where $cum(\bar{A}_{j-1}) = \sum_{l=1}^{j-1} A_l$

Modeling Assumptions

- In this example, parameters are shared across decision points
- Linear regression model for $lCD4_j$ with covariates $lCD4_{j-1}, A_{j-1}, R_j$ and their two- and three-way interactions
- Logistic regression model for R_j with covariates $cum(\bar{A}_{j-1})$ among those with $R_{j-1} = 0$ and $cum(\bar{A}_{j-1}) > 0$

Estimating the Parameters in the CD4 Model

- In this example, parameters are shared across decision points
- We can estimate μ_0 and σ_0^2 using data from the baseline visit
- We can estimate $\beta_0, \dots, \beta_7$ and σ^2 using linear regression and stacking data from multiple time points across multiple subjects together. Specifically, the dataset would have $(M + 1)n$ rows with “response” variable $(lCD4_{1i}, \dots, lCD4_{M+1,i})_{i=1, \dots, n}$. The predictors corresponding to $lCD4_{ji}$ would be $lCD4_{j-1,i}, A_{j-1,i}, R_{ji}$ and their two- and three-way interactions.

Estimating the Parameters in the CD4 Model

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-0.14	0.11	-1.29	0.20
lag_logCD4	0.99	0.02	55.82	0.00
lag_A	1.81	0.15	11.69	0.00
R	-0.01	0.18	-0.07	0.94
lag_logCD4:lag_A	-0.23	0.02	-9.39	0.00
lag_logCD4:R	-0.05	0.03	-1.71	0.09
lag_A:R	-2.21	0.25	-8.89	0.00
lag_logCD4:lag_A:R	0.30	0.04	7.33	0.00

Estimating the Parameters in the Resistance Model

- In this example, parameters are shared across decision points
- The parameters (γ_0, γ_1) can be estimated using standard logistic regression model software
- Outcome would be R_{ji} for all i, j where $\text{cum}(\overline{A}_{j-1,i}) > 0$ and $R_{j-1,i} = 0$

Estimating the Parameters in the Resistance Model

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-3.28	0.14	-22.91	0
lag_cum_A	0.86	0.07	12.30	0

Estimating the Average Response

- Once parameter estimates were obtained then we can estimate $E(Y^{\bar{a}})$ for any $\bar{a}$ by using the Monte Carlo methods previously described
- Estimate the anticipated CD4 and log CD4 at 24 months under the following eight treatment sequences: (0, 0, 0, 0); (0, 0, 0, 1); (0, 0, 1, 1); (0, 1, 1, 1); (1, 1, 1, 1); (1, 1, 1, 0); (1, 1, 0, 0); (1, 0, 0, 0)

Estimating the Average Response

```
mean_response <- function(treat_sequence, coef_estimates, MC) {  
  mu0 <- coef_estimates[1]; sigma0 <- coef_estimates[2]  
  beta0 <- coef_estimates[3]; beta1 <- coef_estimates[4]; beta2 <- coef_estimates[5];  
  beta3 <- coef_estimates[6]; beta4 <- coef_estimates[7];  
  beta5 <- coef_estimates[8]; beta6 <- coef_estimates[9];  
  beta7 <- coef_estimates[10];  
  
  sigma <- coef_estimates[11]; gamma0 <- coef_estimates[12]; gamma1 <- coef_estimates[13]  
  
  R0 <- rep(0, MC)  
  logCD40 <- rnorm(MC, mu0, sigma0)  
  A0 <- rep(treat_sequence[1], MC)  
  cumA0 <- A0  
  R1 <- rbinom(MC, 1, expit(gamma0 + gamma1*cumA0))*(cumA0 > 0 & R0 == 0) +  
    0*(cumA0 == 0) + 1*(R0 == 1)  
  logCD41 <- beta0 + beta1*logCD40 + beta2*A0 + beta3*R1 +  
    beta4*logCD40*A0 + beta5*logCD40*R1 + beta6*A0*R1 + beta7*logCD40*A0*R1 +  
    rnorm(MC, 0, sigma)  
  A1 <- rep(treat_sequence[2], MC)  
  cumA1 <- A0 + A1  
  R2 <- rbinom(MC, 1, expit(gamma0 + gamma1*cumA1))*(cumA1 > 0 & R1 == 0) +  
    0*(cumA1 == 0) + 1*(R1 == 1)  
  logCD42 <- beta0 + beta1*logCD41 + beta2*A1 + beta3*R2 +  
    beta4*logCD41*A1 + beta5*logCD41*R2 + beta6*A1*R2 + beta7*logCD41*A1*R2 +  
    rnorm(MC, 0, sigma)  
  A2 <- rep(treat_sequence[3], MC)  
  cumA2 <- A0 + A1 + A2  
  R3 <- rbinom(MC, 1, expit(gamma0 + gamma1*cumA2))*(cumA2 > 0 & R2 == 0) +  
    0*(cumA2 == 0) + 1*(R2 == 1)  
  logCD43 <- beta0 + beta1*logCD42 + beta2*A2 + beta3*R3 +  
    beta4*logCD42*A2 + beta5*logCD42*R3 + beta6*A2*R3 + beta7*logCD42*A2*R3 +  
    rnorm(MC, 0, sigma)  
  A3 <- rep(treat_sequence[4], MC)  
  cumA3 <- A0 + A1 + A2 + A3
```

Estimating the Average Response

	mean_logCD4	mean_CD4
(0, 0, 0, 0)	5.48	325
(0, 0, 0, 1)	5.90	459
(0, 0, 1, 1)	6.07	557
(0, 1, 1, 1)	6.01	567
(1, 1, 1, 1)	5.76	489
(1, 1, 1, 0)	5.73	460
(1, 1, 0, 0)	5.69	432
(1, 0, 0, 0)	5.59	382

Bootstrap to Estimate SE

	SE_logCD4	SE_CD4
(0, 0, 0, 0)	0.039	10.4
(0, 0, 0, 1)	0.022	8.7
(0, 0, 1, 1)	0.018	9.8
(0, 1, 1, 1)	0.027	15.9
(1, 1, 1, 1)	0.046	23.6
(1, 1, 1, 0)	0.036	17.9
(1, 1, 0, 0)	0.030	13.7
(1, 0, 0, 0)	0.027	11.2

The gfoRmula Package

- A very new R Package for estimating the effects of sustained treatment strategies via the parametric g-formula that can be installed from CRAN:
- More information and examples can be found in the following paper:

McGrath, S., Lin, V., Zhang, Z., Petito, L. C., Logan, R. W., Hernán, M. A., & Young, J. G. (2020). gfoRmula: An R Package for Estimating the Effects of Sustained Treatment Strategies via the Parametric g-formula. *Patterns*, 1(3), 100008. doi:<https://doi.org/10.1016/j.patter.2020.100008>

Recall G-Computation from class

- We have a time varying treatment as opposed to a treatment that was given at a single time point, and we would like to estimate the mean treatment response.
- Traditional methods don't take into account causal effects. What is the problem if only baseline covariates and treatment history are included in a model? What if all the covariate history and treatment history were included?
- With G-computation, we are trying to estimate the distribution of some outcome by modeling the entire longitudinal covariate process.
- This requires fitting a whole bunch of models each of which has to be “right” in order to consistently estimate the distribution of the response under different treatment assignment patterns
- Note that we actually have to correctly specify the correct conditional distribution (not just the conditional mean) for each AND remember L_j (covariates) could be multidimensional

Package Overview

- The gfoRmula R package has many of the capabilities of the GFORMULA SAS macro, as well some overlapping capabilities described for the gformula command in Stata, to implement the parametric g-formula.
- The package allows for:
 - 1 Binary or continuous/multilevel time-varying treatments.
 - 2 Different types of outcomes (survival or continuous/binary end of follow-up).
 - 3 Data with competing events (survival outcomes) and loss to follow-up.
 - 4 Joint interventions on multiple treatments and other options.
- The main function of the package is the `gformula` function, which we will look at in detail today.

Required Structure of the Input Dataset

- The input dataset to the `gformula` function must be an R `data.table`. For all outcome types, the input dataset should contain one record for each follow-up time k for each subject present at baseline (specified by the parameter `id`). The time index k must start at 0 (indexing baseline) and increase in increments of 1. Each column of the dataset will index one of p time-varying covariates $Z_{j,k}$, $j = 1, \dots, p$. Additional columns will contain values of time-fixed baseline confounders (e.g., race, sex) with values repeated on each line k for that subject in the dataset.
- We will be using an example dataset included in the `gfoRmula` package, consisting of 17,500 observations on 2,500 individuals over 7 time points. Each row in the dataset corresponds to the record of one individual at one time point:

Required Structure of the Input Dataset

	id	t0	L1	L2	L3	A	Y
1	1	0	3	0.5774203	8	1	NA
2	1	1	3	-1.6196027	8	1	NA
3	1	2	3	-1.3912145	8	1	NA
4	1	3	2	-1.3814392	8	0	NA
5	1	4	1	0.4641491	8	1	NA
6	1	5	3	-1.5809908	8	1	NA
7	1	6	3	-1.3820358	8	1	-5.339073

Required Structure of the Input Dataset

- The columns in this dataset are:
 - t0** - Time index.
 - id** - Unique identifier for each individual.
 - L1** - Categorical time-varying covariate.
 - L2** - Continuous time-varying covariate.
 - L3** - Continuous baseline covariate. Baseline values are repeated at each time point for each individual.
 - A** - Binary treatment variable.
 - Y** - Continuous outcome of interest. Because this outcome is only defined at the end of follow-up, values of NA are given in all other time points.
- It's very important to have your data in this specific format for the package to work correctly!

Inputs for gformula

Some of the inputs are pretty straightforward:

```
gform_cont_eof <- gformula(obs_data = continuous_eofdata,  
                           id = 'id',  
                           time_name = 't0',  
                           covnames = c('L1', 'L2', 'A'),  
                           covtypes = c('categorical', 'normal'),  
                           outcome_name = 'Y',  
                           outcome_type = 'continuous_eof',  
                           basecovs = c("L3"),  
                           nsimul = 10000, seed = 1234,  
                           ...)
```

Generating Covariate Histories

- The arguments `histories` and `histvars` must be used to specify any desired functions of history that will be used for estimation.
- The precoded functions for `histories` include:
 - ① `lagged` adds a variable to the specified input dataset named `lagi_Zj`, containing the *i*th lag of *Zj* relative to the current follow-up time.
 - ② `cumavg` adds a variable to the specified input dataset named `cumavg_Zj`, which contains the cumulative average of *Zj* up until the current follow-up time.
 - ③ `lagavg` adds a variable to the specified input dataset named `lag_cumavgi_Zj`, which contains the *i*th lag of the cumulative average of *Zj* relative to the current follow-up time.
- For example, from the code below, we are adding the columns of “lag1_A”, “lag2_A”, ..., “lag7_A”, “lag1_L1”, ..., “lag7_L1”, “lag1_L2”, ..., “lag7_L2” to the dataset:

Specifying Covariate and Outcome Models

- Both `covmodels` and `ymodel` take R model statements as inputs.
- The model statement is passed to an appropriate modeling function based on the `covtypes` and `outcome_type` previously specified. (ex. `glm` with gaussian family and identity link for continuous outcomes...)

```
covparams <- list(covmodels = c(L1 ~ lag1_A + lag1_L1 + L3 + t0,
  rcspline.eval(lag1_L2, knots = c(-1, 0, 1)),
  L2 ~ lag1_A + L1 + lag1_L1 + lag1_L2 + L3 + t0,
  A ~ lag1_A + L1 + L2 + lag1_L1 + lag1_L2 + L3 + t0))
ymodel <- Y ~ A + L1 + L2 + lag1_A + lag1_L1 + lag1_L2 + L3
```

Specifying the Interventions

- The arguments `intvars`, `interventions`, and `int_times` are jointly used to define the user-specified treatment interventions to be compared.
- `intvars` is a list of vectors. The number of vector elements of this list should be the number of user-specified interventions of interest.
- `interventions` is a list, whose elements are lists of vectors. Each list in `interventions` specifies a unique intervention on the relevant variable(s) in `intvars`. Each vector contains a function implementing a particular intervention on a single variable, optionally followed by one or more “intervention values” (i.e., integers used to specify the treatment regime).
- `int_descript` is a vector of character strings, each describing an intervention.

Specifying the Interventions

- For example, to compare two static interventions on a binary time-varying covariate A that sets this variable to 0 at all follow-up times versus 1 at all follow-up times:

```
intvars <- list('A', 'A')
interventions <- list(list(c(static, rep(0, 7))), list(c(static, rep(1, 7))))
int_descript <- c('Never treat', 'Always treat')
```

Putting it all together

```
covparams <- list(covmodels = c(L1 ~ lag1_A + lag1_L1 + L3 + t0 +  
  rcspline.eval(lag1_L2, knots = c(-1, 0, 1)),  
  L2 ~ lag1_A + L1 + lag1_L1 + lag1_L2 + L3 + t0,  
  A ~ lag1_A + L1 + L2 + lag1_L1 + lag1_L2 + L3 + t0))  
ymodel <- Y ~ A + L1 + L2 + lag1_A + lag1_L1 + lag1_L2 + L3  
intvars <- list('A', 'A')  
interventions <- list(list(c(static, rep(0, 7))), list(c(static, rep(1, 7))))  
int_descript <- c('Never treat', 'Always treat')  
  
gform_cont_eof <- gformula(obs_data = continuous_eofdata,  
  id = 'id', time_name = 't0',  
  covnames = c('L1', 'L2', 'A'), outcome_name = 'Y',  
  outcome_type = 'continuous_eof',  
  covtypes = c('categorical', 'normal', 'binary'),  
  histories = c(lagged), histvars = list(c('A', 'L1', 'L2')),  
  covparams = covparams, ymodel = ymodel,  
  intvars = intvars, interventions = interventions,  
  int_descript = int_descript,  
  basecovs = c("L3"), nsimul = 10000, seed = 1234)
```

Outputs for gformula

From our example of a continuous end-of-follow-up outcome, we get this output table of estimated mean outcome, mean difference, and mean ratio for all interventions (including natural course) at the last time point:

```
gform_cont_eof
```

```
PREDICTED RISK UNDER MULTIPLE INTERVENTIONS
```

```
Intervention    Description
0      Natural course
1      Never treat
2      Always treat
```

```
Sample size = 2500, Monte Carlo sample size = 10000
Number of bootstrap samples = 0
Reference intervention = natural course (0)
```

k	Interv.	NP mean	g-form mean	Mean ratio	Mean difference	% Intervened	On
6	0	-4.414543	-4.348234	1.000000	0.000000	0.0	
6	1	NA	-3.107835	0.714735	1.2403990	100.0	
6	2	NA	-4.603006	1.058592	-0.2547717	73.7	
Aver % Intervened On							
		0.00000					
		82.03143					
		17.41714					

Bootstrapping

The `gformula` function also allows for easy bootstrapping:

```
#ncores <- parallel::detectCores()-1
gform_cont_eof_b <- gformula(obs_data = continuous_eofdata,
  id = 'id', time_name = 't0',
  covnames = c('L1', 'L2', 'A'), outcome_name = 'Y',
  outcome_type = 'continuous_eof', covtypes = c('categorical', 'normal', 'binary'),
  histories = c(lagged), histvars = list(c('A', 'L1', 'L2')),
  covparams = covparams, ymodel = ymodel,
  intvars = intvars, interventions = interventions,
  int_descript = int_descript,
  basecovs = c("L3"), nsimul = 10000, seed = 1234,
  parallel = FALSE, nsamples = 10,)
```

Note: I'm using 10 bootstrap samples for this example, but you should probably use more in a real case.

Bootstrapped Outputs

The output reports parametric g-formula estimates of the mean outcome (g-form mean) under “never treat”, the “always treat”, and the natural course. It also reports nonparametric estimates of the outcome mean (NP mean) under the natural course, along with 95% confidence intervals for these means and mean differences and ratios comparing each intervention with the natural course:

```
gform_cont_eof_b$result
```

	k	Interv.	NP mean	g-form mean	Mean SE	Mean lower 95% CI	
1:	6	0	-4.414543	-4.352749	0.021597542		-4.400755
2:	6	1	NA	-3.105033	0.489344610		-3.382658
3:	6	2	NA	-4.603008	0.005076661		-4.611692
Mean upper 95% CI Mean ratio MR SE MR lower 95% CI MR upper 95% CI							
1:			-4.342372	1.000000	0.00000000	1.0000000	1.0000000
2:			-1.997202	0.713350	0.10977248	0.4585361	0.7699735
3:			-4.597754	1.057495	0.00527667	1.0458782	1.0593251
Mean difference MD SE MD lower 95% CI MD upper 95% CI % Intervened On							
1:			0.0000000	0.0000000	0.0000000	0.0000000	0.00
2:			1.2477154	0.4746091	1.0105570	2.3578269	100.00
3:			-0.2502596	0.0219698	-0.2579208	-0.2018973	73.83
Aver % Intervened On							
1:			0.00000				
2:			82.13429				
3:			17.43143				

All Available Output

```
str(gform_cont_eof_b)
```

List of 19

\$ result :Classes 'data.table' and 'data.frame': 3 obs. of 17 variables:

```
..$ k : num [1:3] 6 6 6
..$ Interv. : num [1:3] 0 1 2
..$ NP mean : num [1:3] -4.41 NA NA
..$ g-form mean : num [1:3] -4.35 -3.11 -4.6
..$ Mean SE : num [1:3] 0.0216 0.48934 0.00508
..$ Mean lower 95% CI : num [1:3] -4.4 -3.38 -4.61
..$ Mean upper 95% CI : num [1:3] -4.34 -2 -4.6
..$ Mean ratio : num [1:3] 1 0.713 1.057
..$ MR SE : num [1:3] 0 0.10977 0.00528
..$ MR lower 95% CI : num [1:3] 1 0.459 1.046
..$ MR upper 95% CI : num [1:3] 1 0.77 1.06
..$ Mean difference : num [1:3] 0 1.25 -0.25
..$ MD SE : num [1:3] 0 0.475 0.022
..$ MD lower 95% CI : num [1:3] 0 1.011 -0.258
..$ MD upper 95% CI : num [1:3] 0 2.358 -0.202
..$ % Intervened On : num [1:3] 0 100 73.8
..$ Aver % Intervened On: num [1:3] 0 82.1 17.4
.- attr(*, ".internal.selfref")=<externalptr>
```

\$ coeffs :List of 4

```
..$ L1: num [1:2, 1:7] 0.2532 -0.0553 1.2202 1.2073 -0.0752 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:2] "2" "3"
.. .. ..$ : chr [1:7] "(Intercept)" "lag1_A" "lag1_L12" "lag1_L13" ...
..$ L2: Named num [1:9] 0.49329 -1.99819 0.00244 0.00109 0.00278 ...
.. ..- attr(*, "names")= chr [1:9] "(Intercept)" "lag1_A" "L12" "L13" ...
..$ A : Named num [1:10] 0.953 0.569 0.516 1.01 0.301 ...
.. ..- attr(*, "names")= chr [1:10] "(Intercept)" "lag1_A" "L12" "L13" ...
..$ Y : Named num [1:10] -1.7909 -2.007 -0.8 -1.5949 0.0217 ...
.. ..- attr(*, "names")= chr [1:10] "(Intercept)" "A" "L12" "L13" ...
```

Output Regression Models

```
gform_cont_eof_b$coeffs
```

```
$L1
```

```
(Intercept) lag1_A lag1_L12 lag1_L13 L3 t0
2 0.25322813 1.220242 -0.07516617 -0.06843042 0.01878844 0.026644993
3 -0.05532722 1.207326 -0.10497968 -0.09380443 0.01728617 0.009052199
  rcspline.eval(lag1_L2, knots = c(-1, 0, 1))
2 -1.25572843
3 -0.05213063
```

```
$L2
```

```
(Intercept) lag1_A L12 L13 lag1_L12
0.4932864462 -1.9981864775 0.0024354798 0.0010854899 0.0027787028
lag1_L13 lag1_L2 L3 t0
0.0019226034 -0.0006730174 0.0007316841 -0.0010508167
```

```
$A
```

```
(Intercept) lag1_A L12 L13 L2 lag1_L12
0.952502678 0.569425094 0.515880217 1.010480982 0.300525299 -0.027380888
lag1_L13 lag1_L2 L3 t0
-0.021971034 -0.008137860 -0.007159195 -0.008585772
```

```
$Y
```

```
(Intercept) A L12 L13 L2 lag1_A
-1.790947249 -2.006954400 -0.800027604 -1.594912979 0.021723701 0.042776802
lag1_L12 lag1_L13 lag1_L2 L3
0.006353996 0.006983911 0.002501399 -0.002398464
```